COMP 562 Project 1 Report

Cindy Nguyen

Task 1

Q. For numerically calculating the Jacobian in get_jacobian_matrix(), please try with different values of the finite difference (i.e., different values of Δq where q denotes the joint angles). Please report how different values of the finite difference affect the accuracy of the Jacobian-based control. Please also report what is a good value for this finite difference Δq.

As shown in the table below, various finite difference (Δq) values have been tested. The success conditions are less than 0.005-meter positional error and 0.017-radian orientational error. As Δq increases, I observe that the position error becomes larger (worse accuracy). Meanwhile the orientation error seems to fall (better accuracy) to a minimum of around 0.007756 rad and then begins to rise again (worse accuracy). After experimenting, I think a good value for this finite difference is $\Delta q = 0.01$ since it corresponds to the minimum orientation error and its position error is still reasonably low compared to the success criteria, making it the ideal candidate finite difference value.

Δq	Position error (m)	Orientation error (rad)
0.025	0.001827	0.008015
0.02	0.001418	0.008010
0.015	0.001030	0.008003
0.01	**0.000674**	**0.007756**
0.0075	0.000517	0.008019
0.005	0.000349	0.008328
0.0025	0.000308	0.013495

Task 2

Q. Please read the codes of PandaSim.execute() in sim.py (the one you finished in Task 1) and understand how these two checkers (i.e., is state valid() and is state high quality()) are used. Please briefly report how the two checkers will affect the execution of the robot, respectively.

is_state_high_quality(): This checker evaluates if the robot is in a high-quality kinematic posture by calculating the volume-based manipulability from its Jacobian matrix and requiring it to be above 0.01.

Effect on execution: It is executed at every single simulation time step right before the computed joint velocities are sent to the motors. If the state is not high quality (meaning the robot is too close to a kinematic singularity), the execution loop immediately breaks, marking the overall trajectory as invalid (valid = False) and preventing the robot from moving further along that command.

is_state_valid(): This checker evaluates whether the robot's state obeys environmental and task constraints, specifically checking if joint bounds are satisfied, if there are collisions with the ground/obstacles, and if the end-effector remains within the defined 2D workspace boundaries.

Effect on execution: Unlike the high-quality checker, this is evaluated periodically, specifically every 12 simulation steps (which corresponds to every 0.2 seconds in simulation time) after the simulation has stepped. If a collision is detected, joint bounds are violated, or the end-effector leaves the workspace, the execution loop immediately breaks and marks the trajectory as invalid (valid = False), halting further progress.

Task 4

Q. Please show your method and the relevant findings in the report. To show the result, you are encouraged to add figures of the initial and optimized trajectories to your report for comparison.

The optimizer begins with the trajectory returned by the planner, a sequence of rrt.Nod objects each carrying a control and a state. It then performs a local search over that sequence: on every iteration it picks one of the nodes at random and “re-plays” the control attached to it by calling pdef.propagate from the node’s stored state. If propagation succeeds and the resulting state is still valid, the node’s state is replaced with the new one, and the plan’s cost is recomputed.

Cost is defined in terms of end-effector motion: each control contributes its linear-velocity norm times its duration, representing the distance the end-effector will travel. If the updated plan has a lower cost, the change is accepted, otherwise it’s discarded. By repeating this hundreds of times the routine gradually tightens the trajectory, removing unnecessary excursions and smoothing sharp turns, while every candidate remains feasible because it is checked by the simulator.

Overall, the procedure is a hill-climber on the end-effector path length that continually refines node states, yielding shorter, more direct motions without violating dynamics or collision constraints.

Shown in the figures below are the initial path (left) and optimized path (right). We can clearly see that the optimized trajectory is significantly shorter and straighter than the initial.

